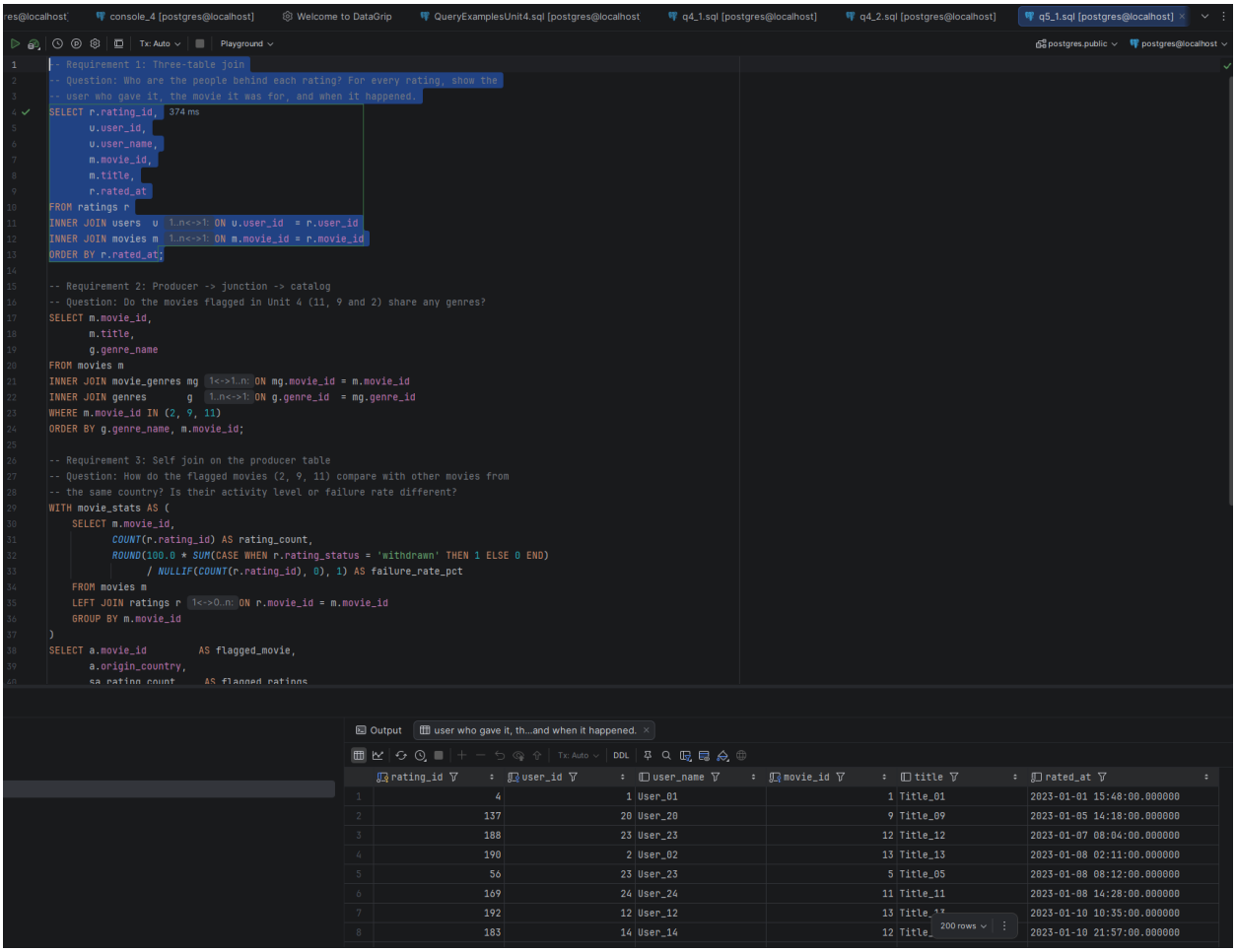
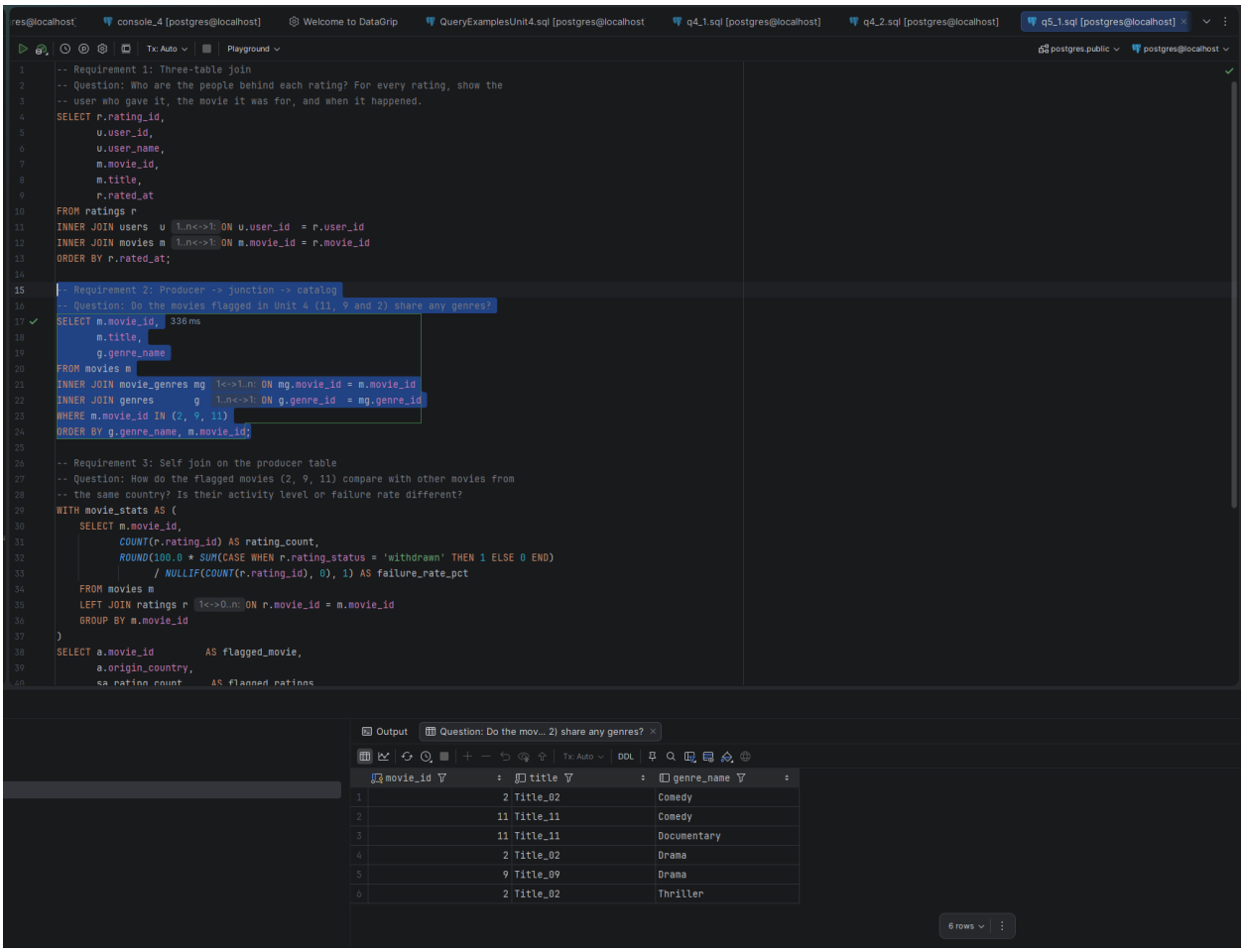
Section 5.1

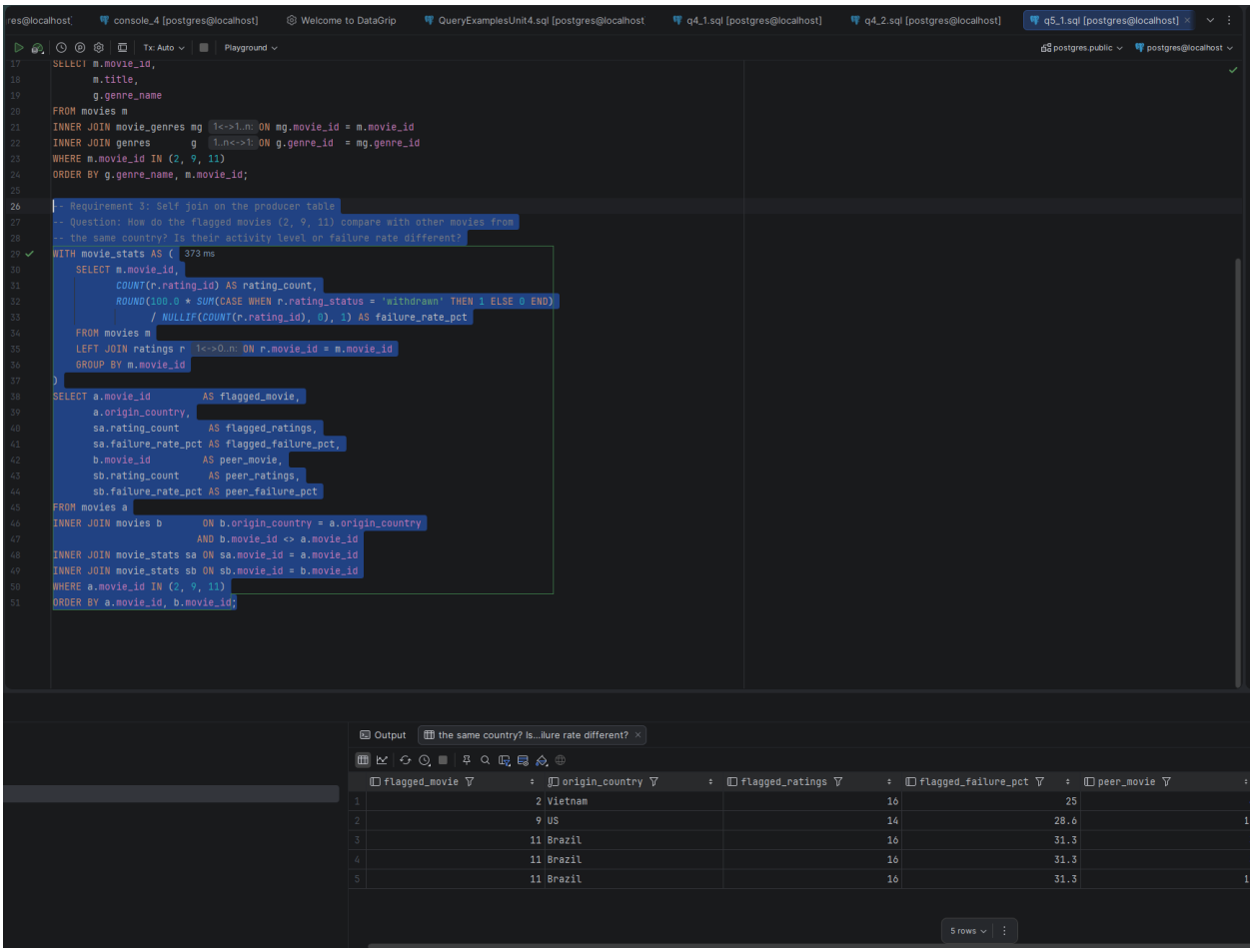
Question 1:



Question 2:


-- Requirement 1: Three-table join
-- Question: Who are the people behind each rating? For every rating, show the
-- user who gave it, the movie it was for, and when it happened.
SELECT r.rating_id,
u.user_id,
u.user_name,
m.movie_id,
m.title,
r.rated_at
FROM ratings r
INNER JOIN users u ON u.user_id = r.user_id
INNER JOIN movies m ON m.movie_id = r.movie_id
ORDER BY r.rated_at;
-- Requirement 2: Producer -> junction -> catalog
-- Question: Do the movies flagged in Unit 4 (11, 9 and 2) share any genres?
SELECT m.movie_id, 336 ms
m.title,
g.genre_name
FROM movies m
INNER JOIN movie_genres mg ON mg.movie_id = m.movie_id
INNER JOIN genres g ON g.genre_id = mg.genre_id
WHERE m.movie_id IN (2, 9, 11)
ORDER BY g.genre_name, m.movie_id;
-- Requirement 3: Self join on the producer table
-- Question: How do the flagged movies (2, 9, 11) compare with other movies from
-- the same country? Is their activity level or failure rate different?
WITH movie_stats AS (
SELECT m.movie_id,
COUNT(r.rating_id) AS rating_count,
ROUND(100.0 * SUM(CASE WHEN r.rating_status = 'withdrawn' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(r.rating_id), 0), 1) AS failure_rate_pct
FROM movies m
LEFT JOIN ratings r ON r.movie_id = m.movie_id
GROUP BY m.movie_id
)
SELECT a.movie_id AS flagged_movie,
a.origin_country,
Output
Question: Do the mov... 2) share any genres?
movie_id
title
genre_name
2 Title_02 Comedy
11 Title_11 Comedy
11 Title_11 Documentary
2 Title_02 Drama
9 Title_09 Drama
2 Title_02 Thriller
6 rows

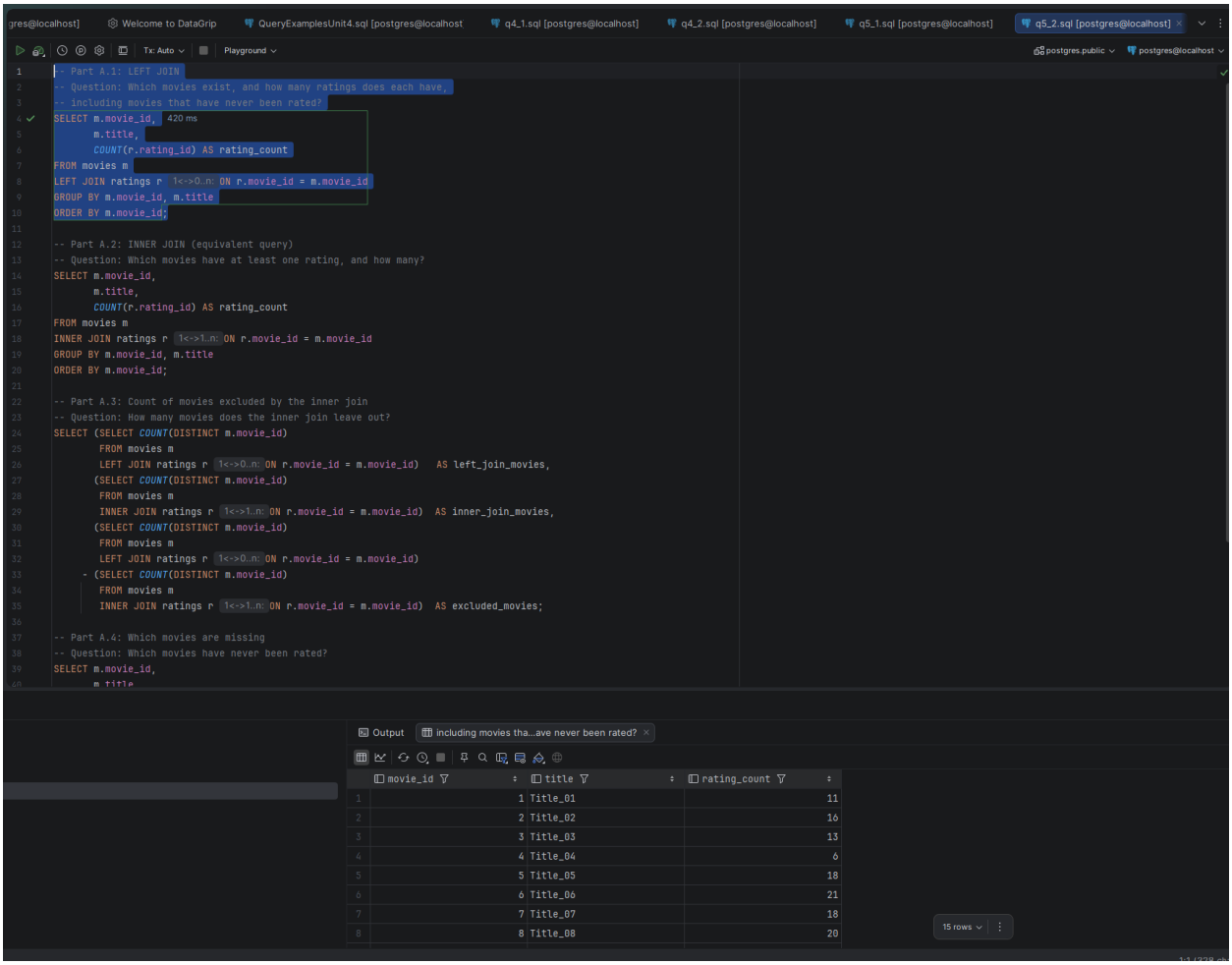
Question 3:


res@localhost
console_4 [postgres@localhost]
Welcome to DataGrip
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
SELECT m.movie_id,
m.title,
g.genre_name
FROM movies m
INNER JOIN movie_genres mg 1<->1..n: ON mg.movie_id = m.movie_id
INNER JOIN genres g 1..n<->1: ON g.genre_id = mg.genre_id
WHERE m.movie_id IN (2, 9, 11)
ORDER BY g.genre_name, m.movie_id;
-- Requirement 3: Self join on the producer table
-- Question: How do the flagged movies (2, 9, 11) compare with other movies from
-- the same country? Is their activity level or failure rate different?
WITH movie_stats AS (373 ms
SELECT m.movie_id,
COUNT(r.rating_id) AS rating_count,
ROUND(100.0 * SUM(CASE WHEN r.rating_status = 'withdrawn' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(r.rating_id), 0), 1) AS failure_rate_pct
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id
)
SELECT a.movie_id AS flagged_movie,
a.origin_country,
sa.rating_count AS flagged_ratings,
sa.failure_rate_pct AS flagged_failure_pct,
b.movie_id AS peer_movie,
sb.rating_count AS peer_ratings,
sb.failure_rate_pct AS peer_failure_pct
FROM movies a
INNER JOIN movies b ON b.origin_country = a.origin_country
AND b.movie_id <> a.movie_id
INNER JOIN movie_stats sa ON sa.movie_id = a.movie_id
INNER JOIN movie_stats sb ON sb.movie_id = b.movie_id
WHERE a.movie_id IN (2, 9, 11)
ORDER BY a.movie_id, b.movie_id;
Output
the same country? Is...illure rate different?
flagged_movie
origin_country
flagged_ratings
flagged_failure_pct
peer_movie
1 2 Vietnam 16 25
2 9 US 14 28.6 1
3 11 Brazil 16 31.3
4 11 Brazil 16 31.3
5 11 Brazil 16 31.3 1
5 rows

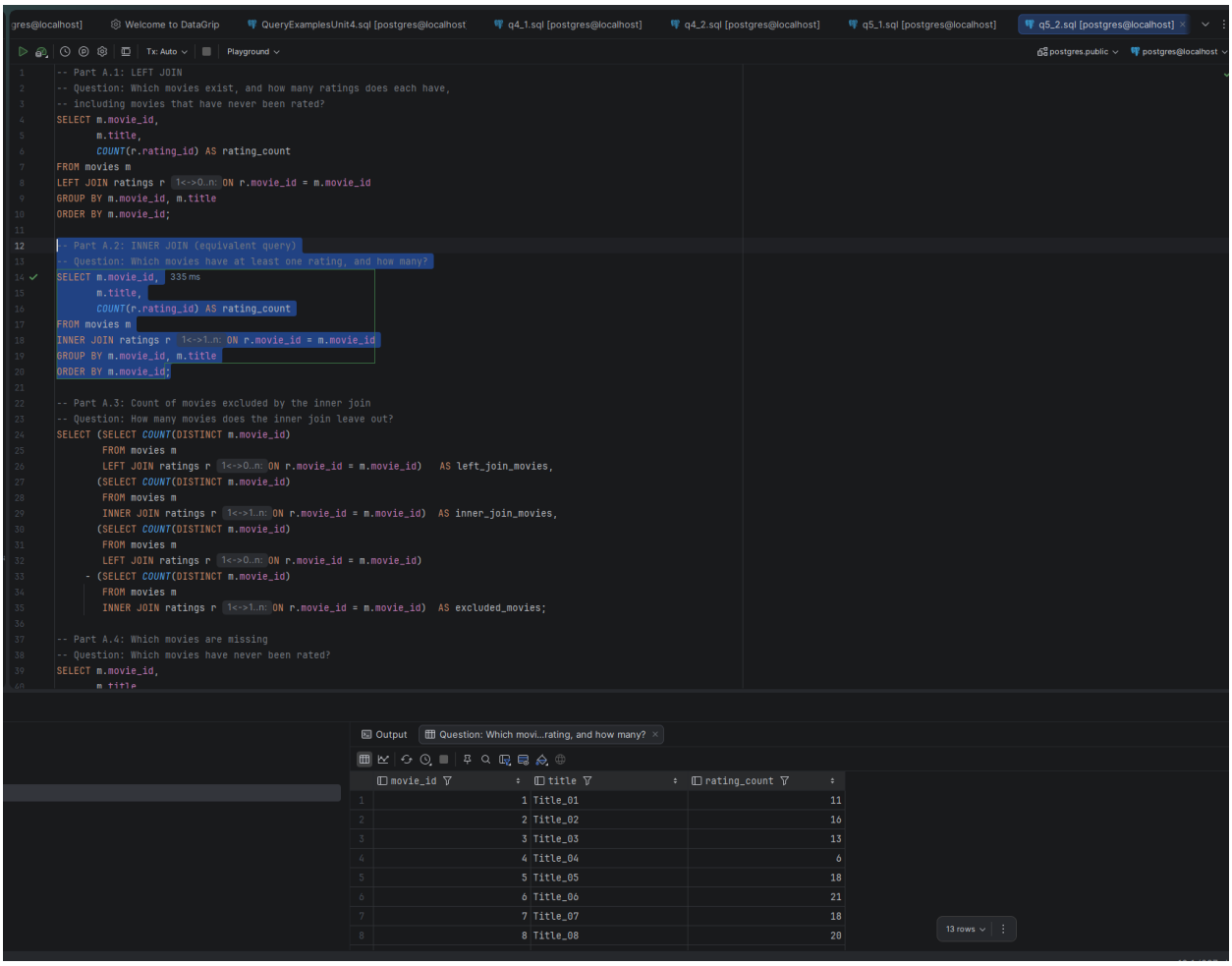
Section 5.2.

Part A

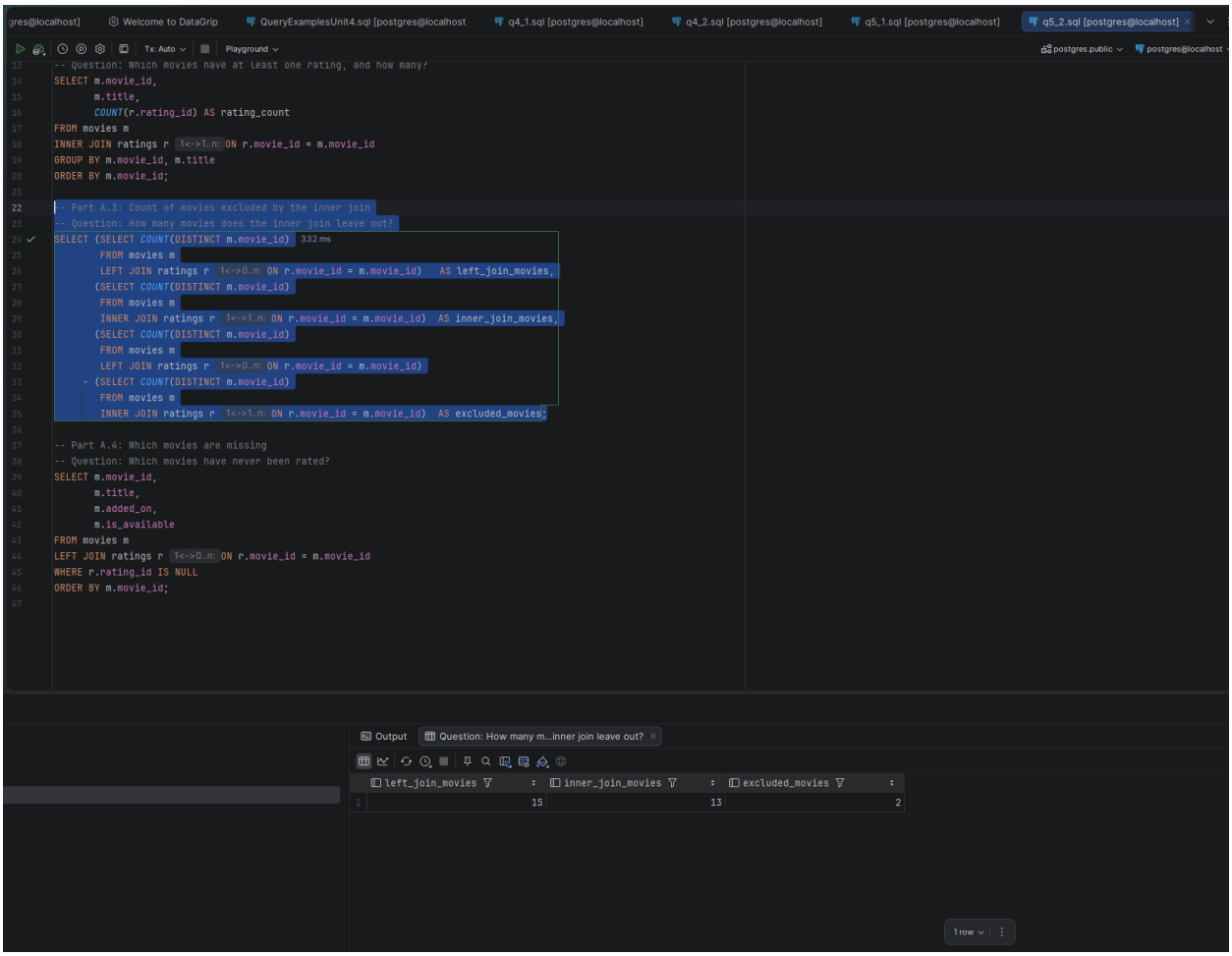
Question 1:



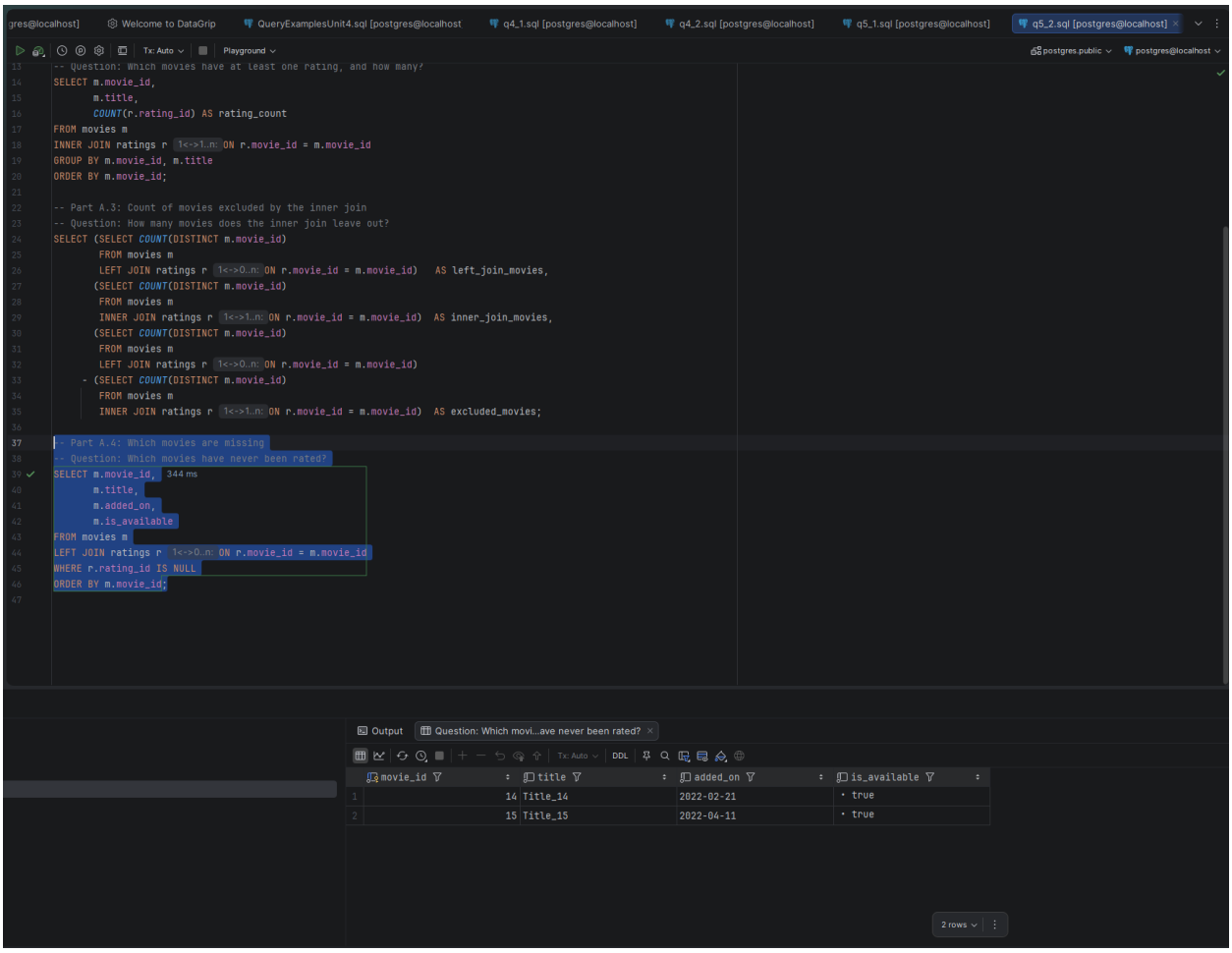
Question 2:


gres@localhost
Welcome to DataGrip
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
q5_2.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
-- Part A.1: LEFT JOIN
-- Question: Which movies exist, and how many ratings does each have,
-- including movies that have never been rated?
SELECT m.movie_id,
m.title,
COUNT(r.rating_id) AS rating_count
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id, m.title
ORDER BY m.movie_id;
-- Part A.2: INNER JOIN (equivalent query)
-- Question: Which movies have at least one rating, and how many?
SELECT m.movie_id, 335 ms
m.title,
COUNT(r.rating_id) AS rating_count
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id, m.title
ORDER BY m.movie_id;
-- Part A.3: Count of movies excluded by the inner join
-- Question: How many movies does the inner join leave out?
SELECT (SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id) AS left_join_movies,
(SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id) AS inner_join_movies,
(SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id)
- (SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id) AS excluded_movies;
-- Part A.4: Which movies are missing
-- Question: Which movies have never been rated?
SELECT m.movie_id,
m.title
Output
Question: Which movi...rating, and how many?
movie_id
title
rating_count
1 Title_01 11
2 Title_02 16
3 Title_03 13
4 Title_04 6
5 Title_05 18
6 Title_06 21
7 Title_07 18
8 Title_08 20
13 rows

Question 3:

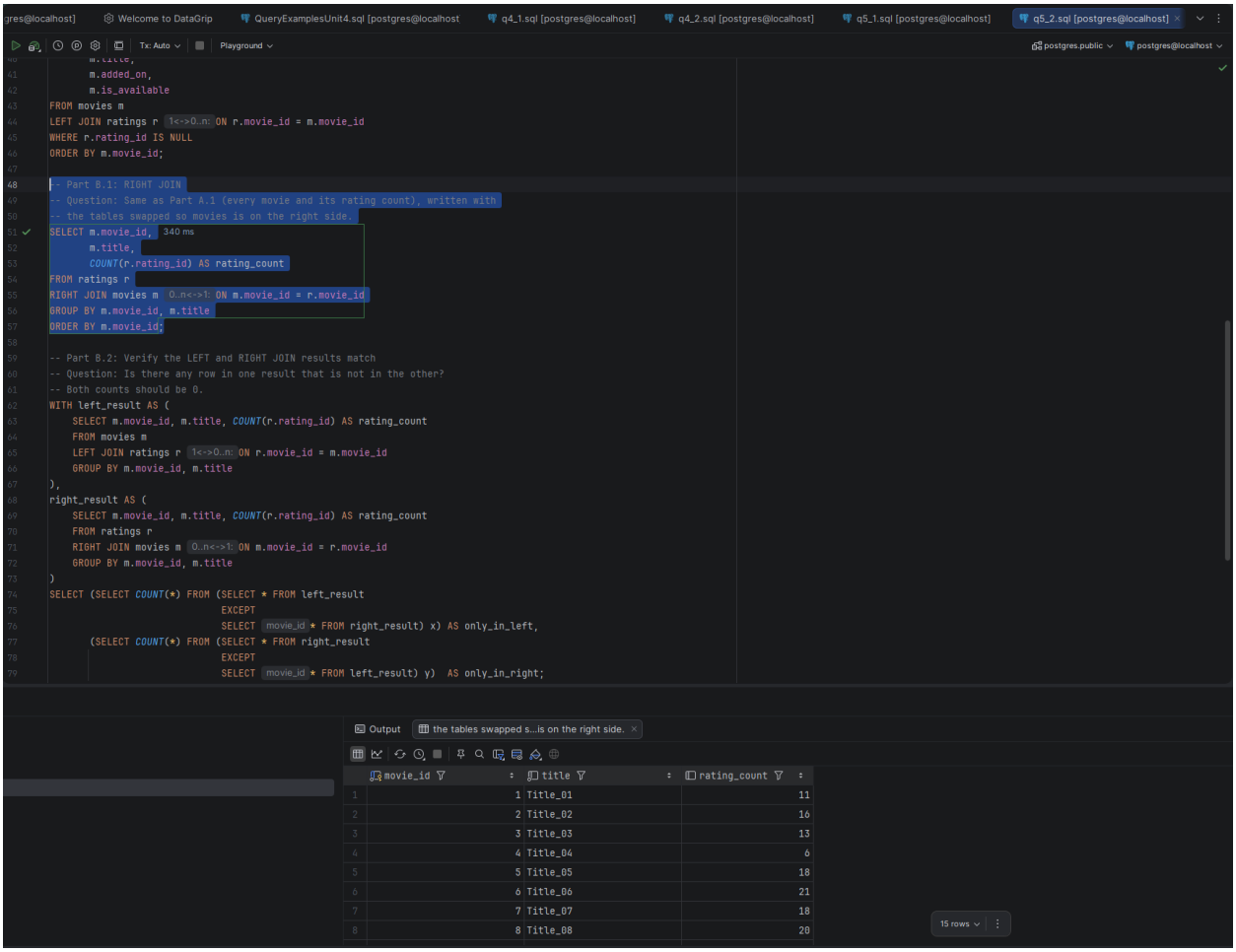

gres@localhost]
Welcome to DataGrip
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
q5_2.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
-- Question: Which movies have at least one rating, and how many?
SELECT m.movie_id,
m.title,
COUNT(r.rating_id) AS rating_count
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id, m.title
ORDER BY m.movie_id;
-- Part A.3: Count of movies excluded by the inner join
-- Question: How many movies does the inner join leave out?
SELECT (SELECT COUNT(DISTINCT m.movie_id) 332 ms
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id) AS left_join_movies,
(SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id) AS inner_join_movies,
(SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id)
- (SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id) AS excluded_movies;
-- Part A.4: Which movies are missing
-- Question: Which movies have never been rated?
SELECT m.movie_id,
m.title,
m.added_on,
m.is_available
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
WHERE r.rating_id IS NULL
ORDER BY m.movie_id;
Output
Question: How many m...inner join leave out?
left_join_movies
inner_join_movies
excluded_movies
1
15
13
2
1 row

Question 4:

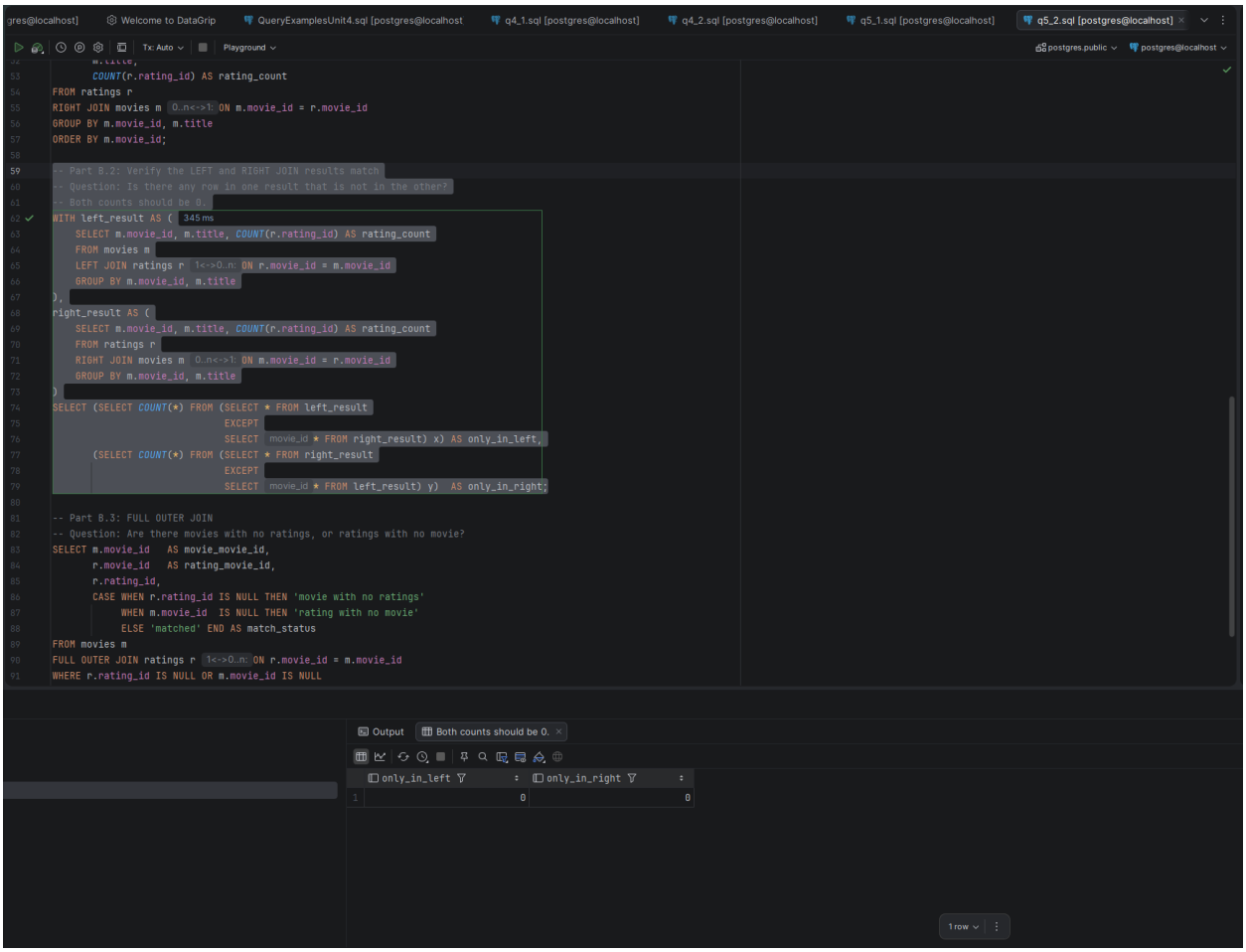

gres@localhost]
Welcome to DataGrip
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
q5_2.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
-- Question: Which movies have at least one rating, and how many?
SELECT m.movie_id,
m.title,
COUNT(r.rating_id) AS rating_count
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id, m.title
ORDER BY m.movie_id;
-- Part A.3: Count of movies excluded by the inner join
-- Question: How many movies does the inner join leave out?
SELECT (SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id) AS left_join_movies,
(SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id) AS inner_join_movies,
(SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id)
- (SELECT COUNT(DISTINCT m.movie_id)
FROM movies m
INNER JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id) AS excluded_movies;
-- Part A.4: Which movies are missing
-- Question: Which movies have never been rated?
SELECT m.movie_id, 344 ms
m.title,
m.added_on,
m.is_available
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
WHERE r.rating_id IS NULL
ORDER BY m.movie_id;
Output
Question: Which movi...ave never been rated?
Tx: Auto
DDL
movie_id
title
added_on
is_available
1 14 Title_14 2022-02-21 true
2 15 Title_15 2022-04-11 true
2 rows

Part B

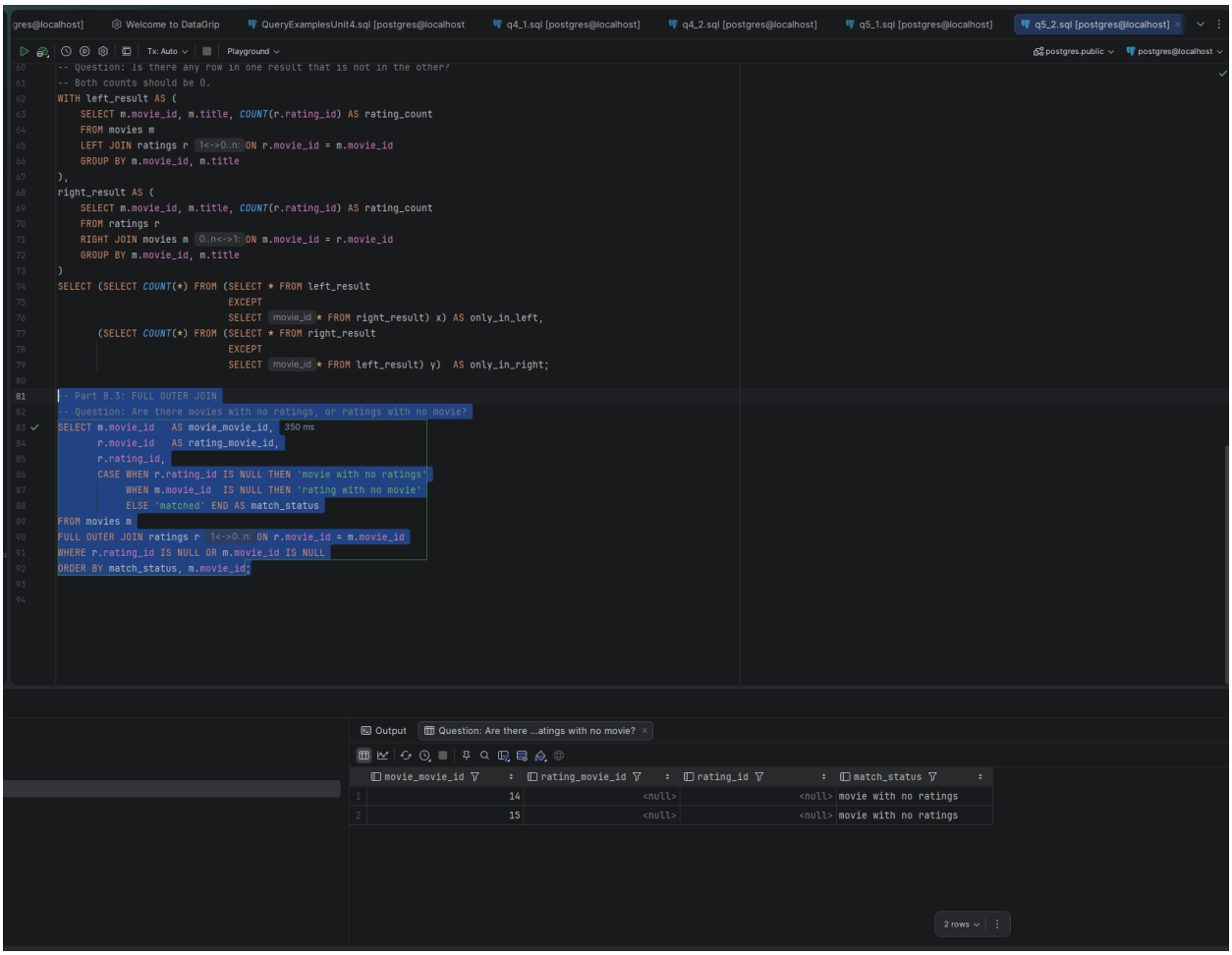
Question 1:



Question 2:


gres@localhost]
Welcome to DataGrip
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
q5_2.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
m.title,
COUNT(r.rating_id) AS rating_count
FROM ratings r
RIGHT JOIN movies m 0..n<->1 ON m.movie_id = r.movie_id
GROUP BY m.movie_id, m.title
ORDER BY m.movie_id;
-- Part B.2: Verify the LEFT and RIGHT JOIN results match
-- Question: Is there any row in one result that is not in the other?
-- Both counts should be 0.
WITH left_result AS (345 ms
SELECT m.movie_id, m.title, COUNT(r.rating_id) AS rating_count
FROM movies m
LEFT JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
GROUP BY m.movie_id, m.title
),
right_result AS (
SELECT m.movie_id, m.title, COUNT(r.rating_id) AS rating_count
FROM ratings r
RIGHT JOIN movies m 0..n<->1: ON m.movie_id = r.movie_id
GROUP BY m.movie_id, m.title
)
SELECT (SELECT COUNT(*) FROM (SELECT * FROM left_result
EXCEPT
SELECT movie_id * FROM right_result) x) AS only_in_left,
(SELECT COUNT(*) FROM (SELECT * FROM right_result
EXCEPT
SELECT movie_id * FROM left_result) y) AS only_in_right;
-- Part B.3: FULL OUTER JOIN
-- Question: Are there movies with no ratings, or ratings with no movie?
SELECT m.movie_id AS movie_movie_id,
r.movie_id AS rating_movie_id,
r.rating_id,
CASE WHEN r.rating_id IS NULL THEN 'movie with no ratings'
WHEN m.movie_id IS NULL THEN 'rating with no movie'
ELSE 'matched' END AS match_status
FROM movies m
FULL OUTER JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
WHERE r.rating_id IS NULL OR m.movie_id IS NULL
Output
Both counts should be 0.
only_in_left
only_in_right
1
0
0
1 row

Question 3:

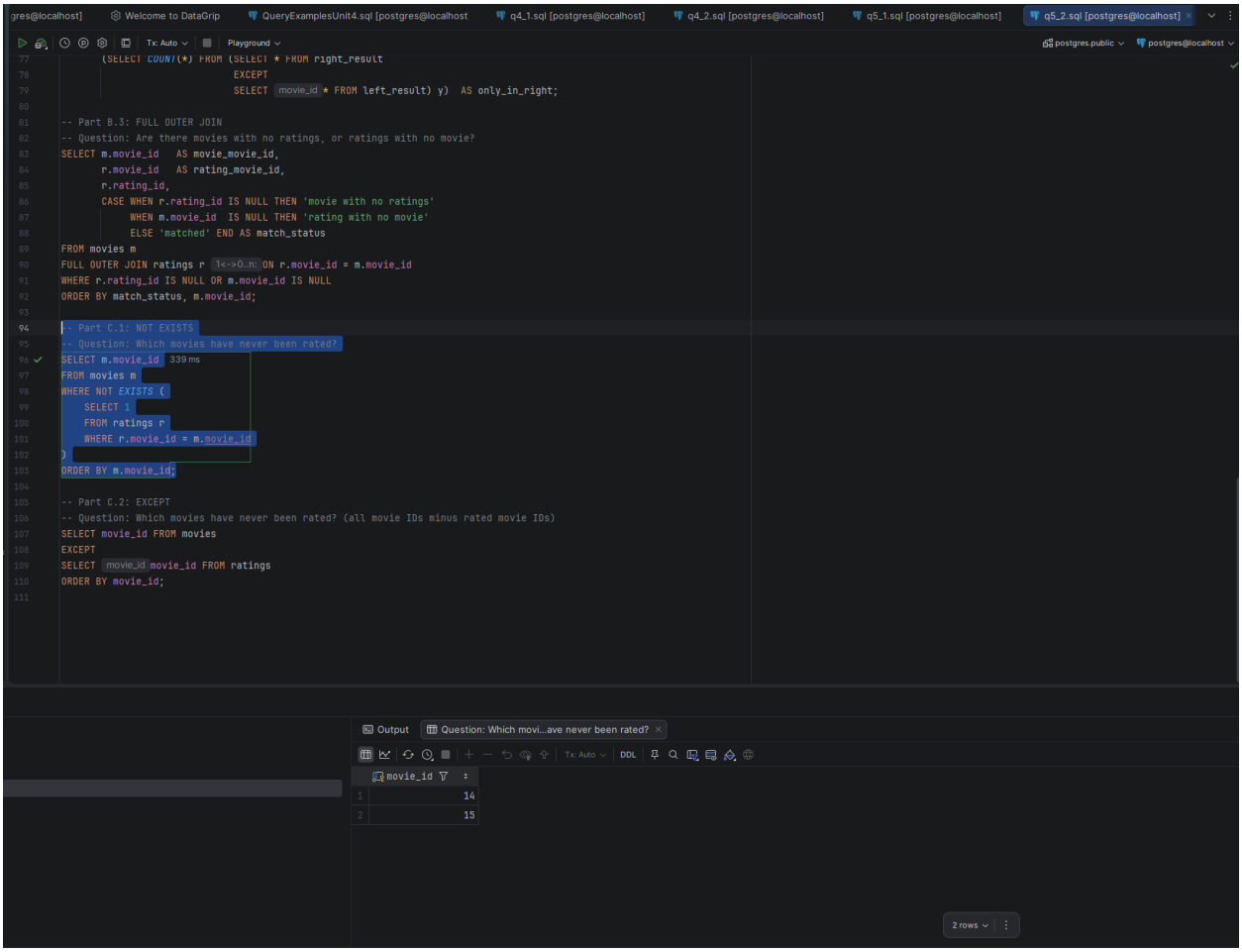


Question 4:

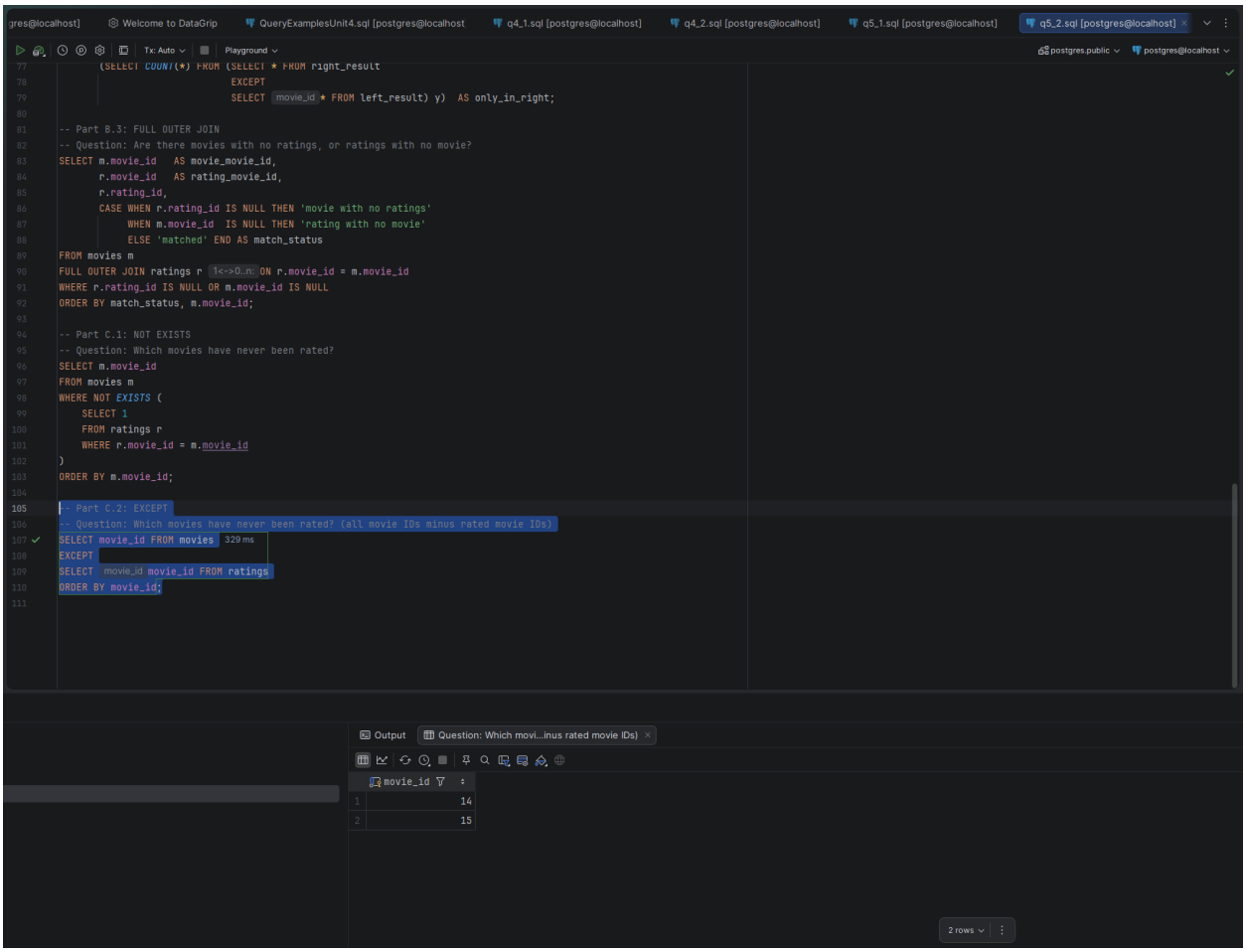
What a full outer join reveals: it checks for gaps on both sides in one query. A left join only shows movies with no ratings. A right join only shows ratings with no movie, the kind of orphan records that bad imports or deleted movies can leave behind. Here there are no orphan ratings because the foreign key fk_ratings_prod prevents them, but a full outer join would catch them in tables without that protection.

Part C:

Question 1:



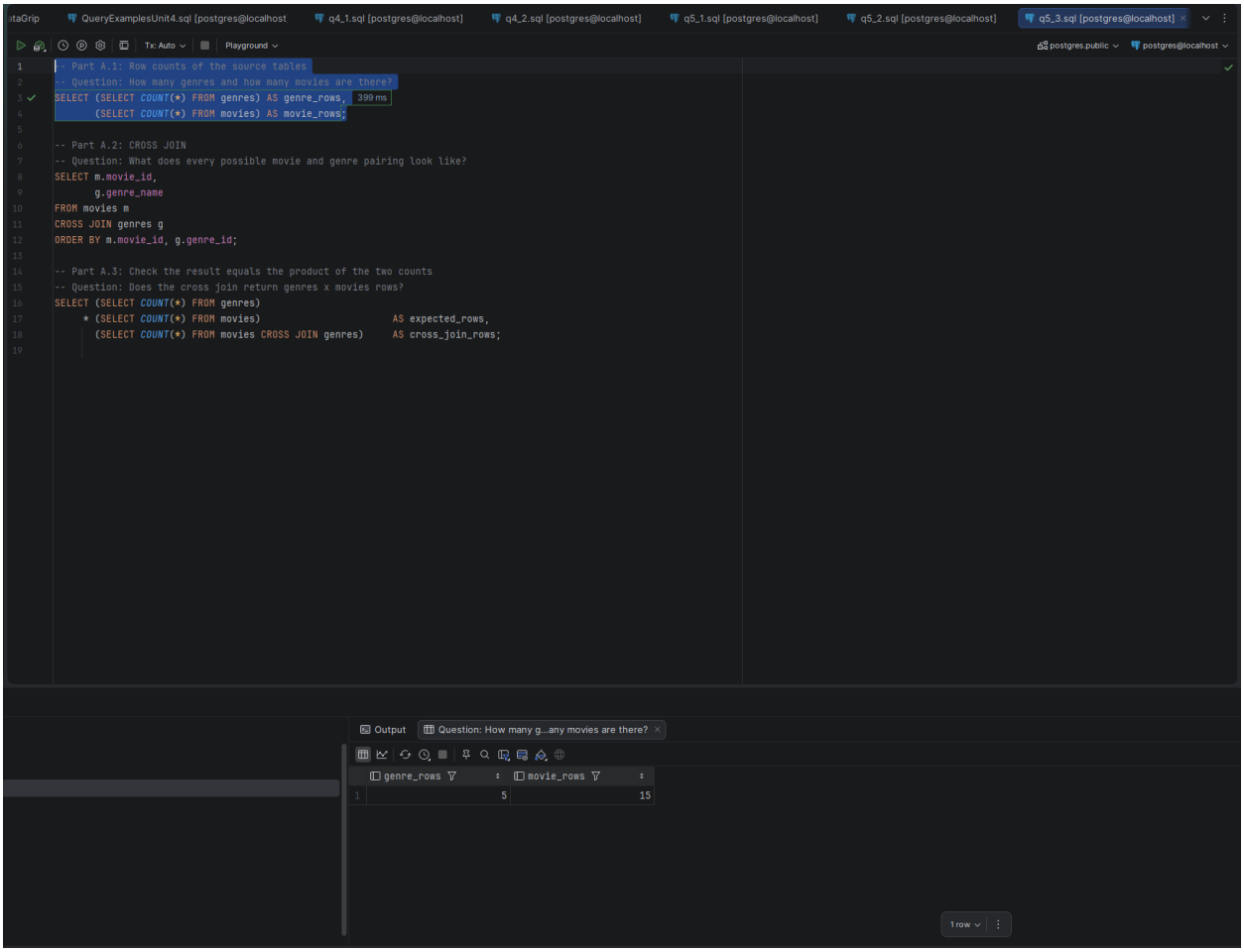
Question 2:


gres@localhost]
Welcome to DataGrip
QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
q5_2.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
(SELECT COUNT(*) FROM (SELECT * FROM right_result
EXCEPT
SELECT movie_id, * FROM left_result) y) AS only_in_right;
-- Part B.3: FULL OUTER JOIN
-- Question: Are there movies with no ratings, or ratings with no movie?
SELECT m.movie_id AS movie_movie_id,
r.movie_id AS rating_movie_id,
r.rating_id,
CASE WHEN r.rating_id IS NULL THEN 'movie with no ratings'
WHEN m.movie_id IS NULL THEN 'rating with no movie'
ELSE 'matched' END AS match_status
FROM movies m
FULL OUTER JOIN ratings r 1<->0..n: ON r.movie_id = m.movie_id
WHERE r.rating_id IS NULL OR m.movie_id IS NULL
ORDER BY match_status, m.movie_id;
-- Part C.1: NOT EXISTS
-- Question: Which movies have never been rated?
SELECT m.movie_id
FROM movies m
WHERE NOT EXISTS (
SELECT 1
FROM ratings r
WHERE r.movie_id = m.movie_id
)
ORDER BY m.movie_id;
-- Part C.2: EXCEPT
-- Question: Which movies have never been rated? (all movie IDs minus rated movie IDs)
SELECT movie_id FROM movies 329 ms
EXCEPT
SELECT movie_id movie_id FROM ratings
ORDER BY movie_id;
Output
Question: Which movi...inus rated movie IDs)
movie_id
1 14
2 15
2 rows

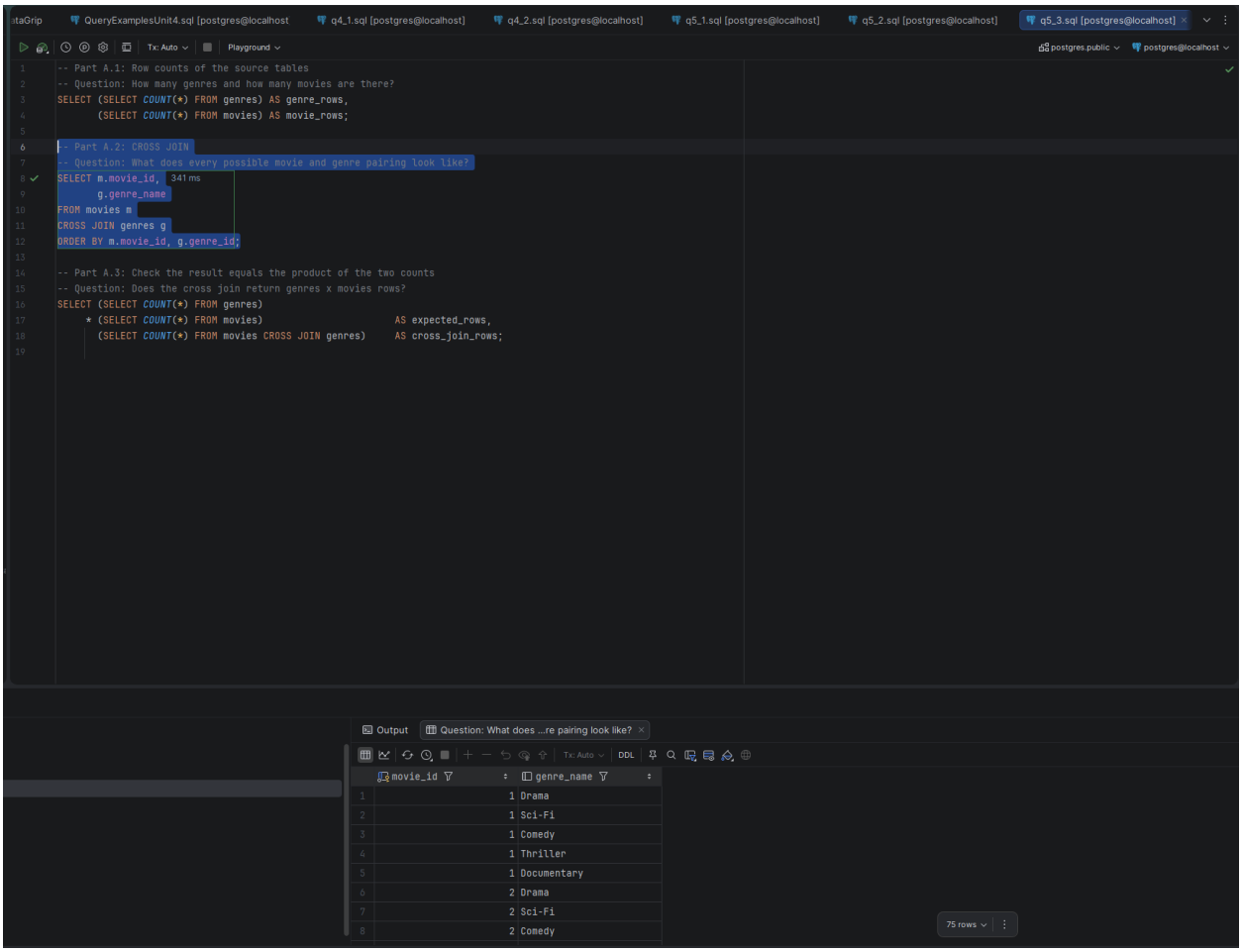
Section 5.3

Part A

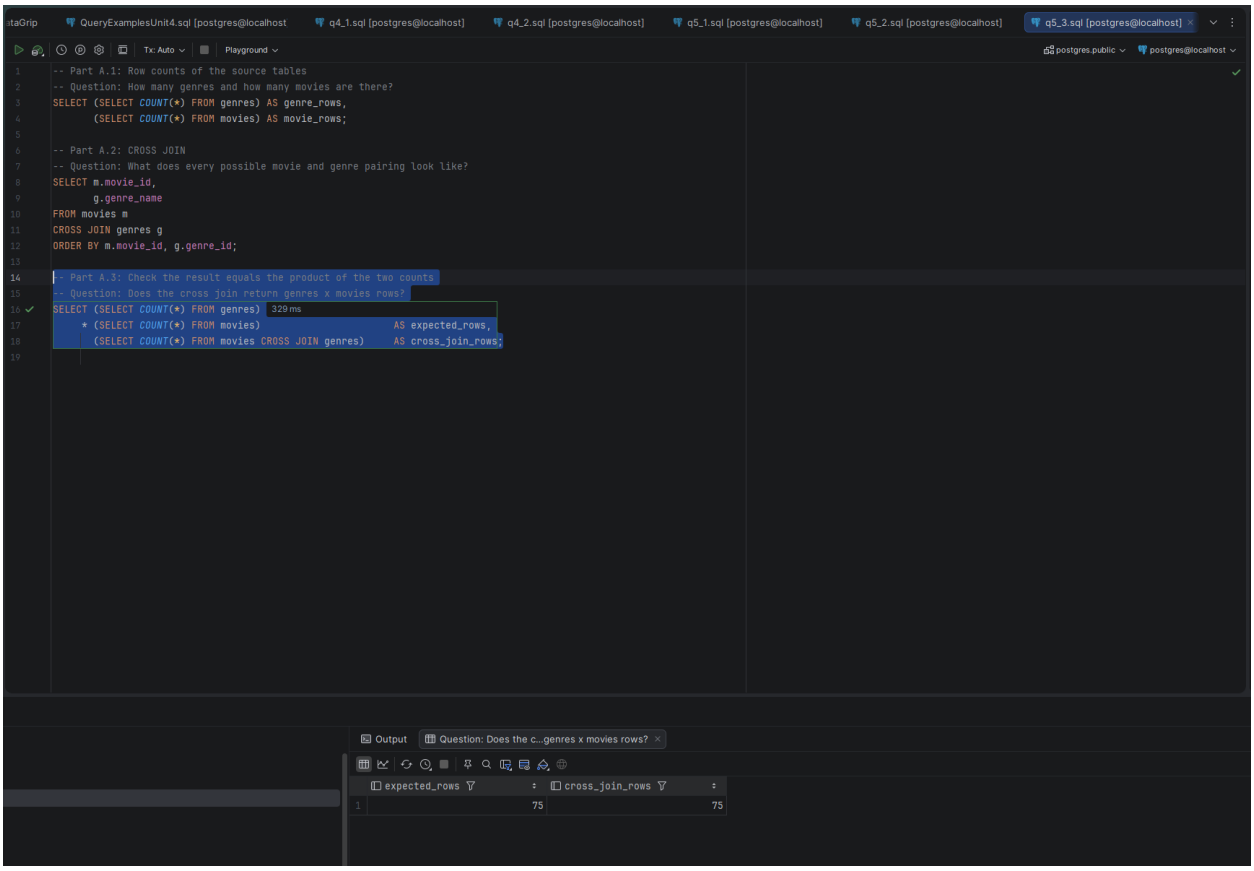
Question 1:



Question 2:


QueryExamplesUnit4.sql [postgres@localhost
q4_1.sql [postgres@localhost]
q4_2.sql [postgres@localhost]
q5_1.sql [postgres@localhost]
q5_2.sql [postgres@localhost]
q5_3.sql [postgres@localhost]
Tx: Auto
Playground
postgres.public
postgres@localhost
-- Part A.1: Row counts of the source tables
-- Question: How many genres and how many movies are there?
SELECT (SELECT COUNT(*) FROM genres) AS genre_rows,
(SELECT COUNT(*) FROM movies) AS movie_rows;
-- Part A.2: CROSS JOIN
-- Question: What does every possible movie and genre pairing look like?
SELECT m.movie_id, 341 ms
g.genre_name
FROM movies m
CROSS JOIN genres g
ORDER BY m.movie_id, g.genre_id;
-- Part A.3: Check the result equals the product of the two counts
-- Question: Does the cross join return genres x movies rows?
SELECT (SELECT COUNT(*) FROM genres)
* (SELECT COUNT(*) FROM movies) AS expected_rows,
(SELECT COUNT(*) FROM movies CROSS JOIN genres) AS cross_join_rows;
Output
Question: What does ...re pairing look like?
Tx: Auto
DDL
movie_id
genre_name
1 1 Drama
2 1 Sci-Fi
3 1 Comedy
4 1 Thriller
5 1 Documentary
6 2 Drama
7 2 Sci-Fi
8 2 Comedy
75 rows

Question 3:



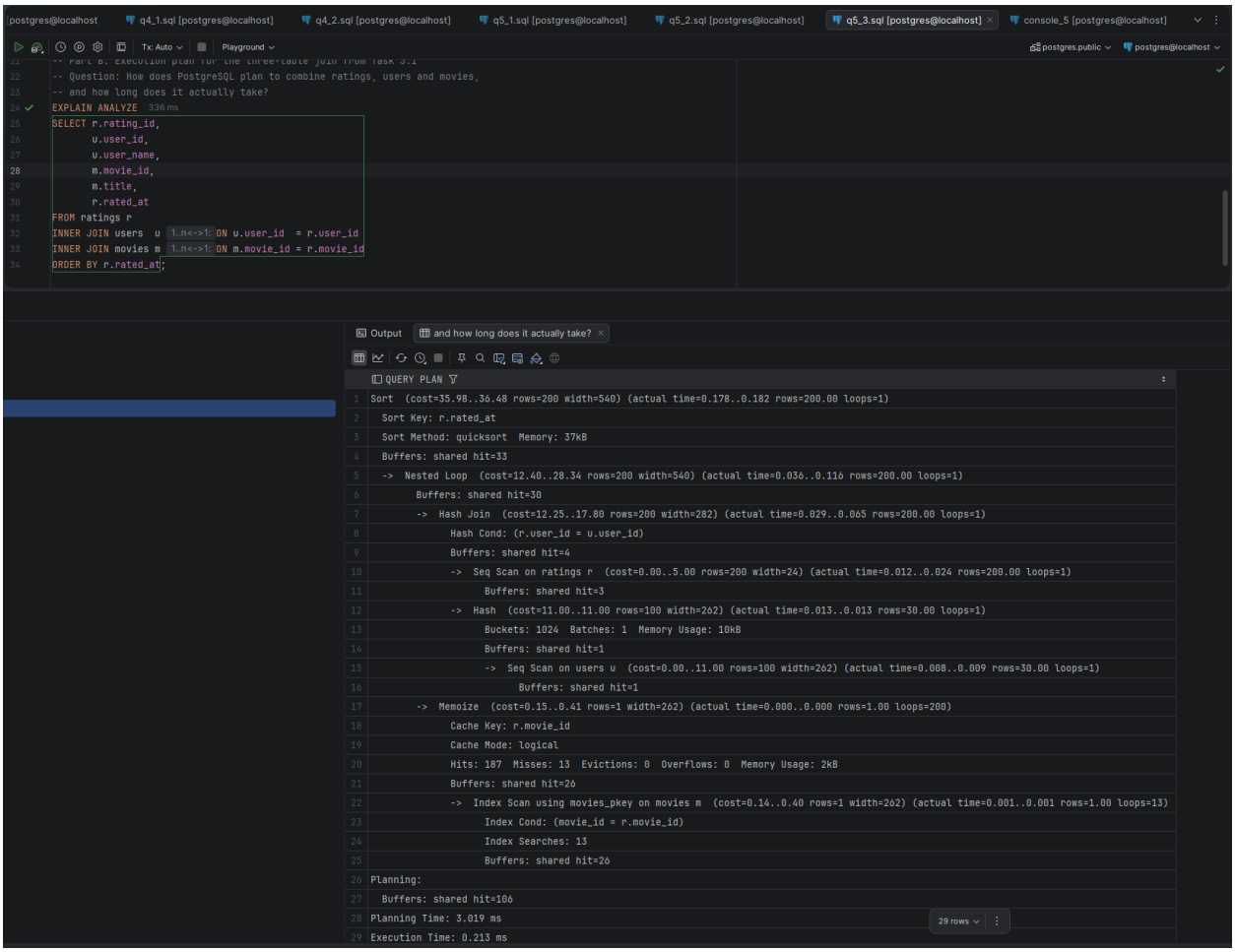
Question 4:

When a Cartesian product is useful: when you need every possible combination on purpose. For example, a movie × genre grid can be compared with movie_genres to find tags that are missing, and a month × region grid makes regions with zero ratings show up as 0 instead of disappearing.

When it signals an error: when the result has far more rows than expected. That usually means a join is missing its ON condition, or a table was listed in FROM without being joined. For example, ratings × users would produce 200 × 30 = 6,000 rows instead of 200, and every total built on it would be inflated.

Part B:

1. Explain analyze query:



2. Join algorithms used: two different ones.

Hash Join for ratings → users. PostgreSQL reads all 30 users into an in-memory hash table (10 kB), then checks each of the 200 ratings against it.

Nested Loop with Memoize for the result → movies. For each rating, it looks up the movie through the primary-key index (movies_pkey). Memoize caches each movie after its first lookup, so it took only 13 index lookups (misses) and 187 cache hits, one miss for each distinct rated movie.

Sort: a final quicksort on rated_at, done entirely in memory (37 kB).

3. Why PostgreSQL chose these:

ratings → users: ratings.user_id has no index and both tables are small, so reading them in full and joining through a hash table is the cheapest option.

→ movies: movies.movie_id does have an index (its primary key), and only 13 distinct movies appear in ratings. With Memoize, nearly every lookup is a cache hit, which costs less than building a second hash table.

Planner estimates: it expected 100 users but found 30. The tables haven't been analyzed since loading, so PostgreSQL used default estimates. Running ANALYZE; would give it accurate counts, and it might then pick a different plan.

Timing: planning took 3.0 ms but execution only 0.2 ms. At this size, deciding how to run the query costs more than running it.

4. What gets slower as the dataset grows:

Seq Scan on ratings: this is the main risk. The query reads every rating, so its cost grows directly with the table.

Sort on rated_at: it fits in 37 kB today. With millions of rows it will exceed work_mem and spill to disk. Watch for Sort Method: external merge. An index on ratings(rated_at) would avoid the sort.

Memoize on movies: it works well because only 13 distinct movies are rated. With thousands of movies the cache would miss far more often, and PostgreSQL would probably switch to a Hash Join.

Hash on users: as users grow, the hash table may no longer fit in memory. When Batches goes above 1, it's writing to disk.